Week 6 Assignment 2 Discussion

From Requirements to Software Models

Central Question - From Requirements to Software Models

Central question: How do we transform confirmed requirements into a defensible software design?

Week 6 outcome:

SmartCare v0.3 — CRC cards, UML, design rationale, Python skeletons and AI design-review record.

Week 6 Learning Journey

Requirements → Domain concepts → Responsibilities → CRC cards → UML → Python skeletons

AI role: design collaborator. Human role: design authority.

Every concept, relationship and responsibility should be traceable to confirmed requirements.

Why Model Before Coding?

Coding too early locks in assumptions and creates accidental design.

Modelling exposes missing concepts, clarifies responsibilities and creates a shared language.

Abstraction means focusing on what matters and deliberately ignoring irrelevant detail.

From Requirements to Domain Concepts

1. Read confirmed requirements and business rules
2. Identify candidate concepts (nouns)
3. Identify behaviours/responsibilities (verbs)
4. Identify relationships and multiplicity
5. Reject concepts with no requirement support

SmartCare candidates: Patient, Practitioner, Appointment. Clinic requires justification.

What Is a Domain Model?

A conceptual representation of important domain concepts, their responsibilities and relationships.

It is independent of UI and database details.

SmartCare core concepts: Patient • Practitioner • Appointment.

Class, Object, State and Behaviour

Class = type | Object = instance | Responsibility = what it knows or does

State → attributes: patient ID, name, specialty, date/time, status

Behaviour → operations: cancel(), validate(), state changes

Place behaviour with the object that has the appropriate responsibility.

Responsibility-Driven Design + CRC Cards

Ask: Who should know this? Who should do this?

CRC = Class • Responsibilities • Collaborators

Patient: identity/state + basic validation → collaborates with Appointment
Appointment: patient + practitioner + time + status + state changes
Practitioner: identity/specialty → collaborates with Appointment

Collaborators, Cohesion and Coupling

Objects collaborate to fulfil requirements, and collaborations reveal dependencies.

Healthy design: narrow, meaningful dependencies and related responsibilities kept together.

Warning signs: too many collaborators, unrelated tasks in one class, or every class knowing every other clas

Relationships and Multiplicity

Association = meaningful relationship between domain concepts.

Multiplicity: 1 = exactly one | 0..1 = optional one | 0..* = zero or many | 1..* = one or many

SmartCare: Patient 1 ↔ 0..* Appointments;
Practitioner 1 ↔ 0..* Appointments;
each Appointment has exactly one Patient and Practitioner.

Association, Aggregation, Composition and Inheritance

Association: related concepts collaborate.
Aggregation: weak whole–part.
Composition: strong ownership/lifecycle — use only when domain meaning supports it.
Inheritance: genuine is-a relationship — not merely code reuse.

For SmartCare, Patient ↔ Appointment is best treated as an association, not composition.

UML Class Diagrams: The Essentials

A UML class diagram communicates:

- class name
- attributes
- operations
- relationships
- multiplicity

Visibility: + public | – private | # protected
Python conventions differ; UML communicates design intent.

SmartCare Initial UML + Traceability

Patient 1 ——— 0..* Appointment
Practitioner 1 – 0..* Appointment

Appointment has exactly one Patient and one Practitioner.

Trace requirements into design:
FR-04 Create appointment → booking responsibility
FR-06 Cancel appointment → Appointment state responsibility.

Design Quality: Avoid the God Class

God class: one class handles patients, appointments, database, UI and reports.

Low cohesion: unrelated tasks in one class; weak conceptual focus.
High coupling: every class knows every other class; changes ripple widely.

Prefer focused responsibilities and narrow dependencies.

Good Responsibility Allocation

Patient = patient state and basic validation
Practitioner = practitioner state/specialty
Appointment = booking state and legal state changes

Services later coordinate cross-object workflows.
Repositories later persist data.

Do not push UI/database/workflow concerns into domain classes.

AI as Design Collaborator

AI can help: extract candidate concepts, spot relationships,
offer alternatives, explain UML and generate critique questions.

Human engineer decides what belongs in the model and must justify it from requirements.

What AI Commonly Gets Wrong

Common modelling failures:

- every noun becomes a class
- Manager/Controller classes everywhere
- unnecessary inheritance
- invented features
- over-engineering

Typical overdesign: PatientManager, AppointmentManager, ClinicController, ScheduleEngine, NotificationManager.

Critique Before Accepting an AI Design

Ask four questions:
1. Does this represent the confirmed domain?
2. Does each class have a clear responsibility?
3. Is each relationship supported by evidence?
4. Is the added complexity justified?

Reject plausible-looking design that lacks requirement support.

Engineering Prompt for Domain Modelling

Use only the confirmed SmartCare requirements and business rules.
Suggest candidate classes, responsibilities, collaborators and relationships.
For every proposed element, cite the supporting requirement.
Do not invent features, database concerns or UI details.
Identify uncertain proposals separately for human review.

Human Model First → AI Alternative → Decision

1. Create the human model first.
2. Ask AI for an alternative or critique.
3. Compare the two models against requirements.
4. Record Accept / Modify / Reject decisions.

AI is most valuable when it creates a design conversation — not when it silently replaces design reasoning.

From UML to Python Skeletons

Approved model → Python class skeletons.

Implementation should not silently redefine the design.

Week 3 skeletons may intentionally contain placeholders: the goal is structural consistency, not full behaviour.

Model–Code Consistency

Check that:

- class names align with UML
- attributes reflect modelled state
- methods reflect responsibilities
- relationships are represented consistently
- no unsupported classes/features appear during coding

If implementation forces a design change, update the model deliberately.

SmartCare v0.3 Deliverable + Design Checklist

Deliverables:

- CRC cards
- UML class diagram
- design rationale
- Python skeletons
- AI design-review record

Checklist: clear purpose, defensible relationships/multiplicity, no unsupported features, explainable model.

Week 6 Rules

ENGINEERING RULE
Model the problem before coding the solution.

AI RULE
AI may propose designs; humans decide responsibilities.

A good model is not the most complicated model — it is the simplest model that accurately represents the confirmed problem.

Next Week: Object-Oriented Python Design

Week 8 question: What makes an object-oriented design good?

We move from model → implementation with:

- encapsulation
- composition
- inheritance (when justified)
- polymorphism
- validation and controlled state changes